AVANCE DE PROYECTO · AULAS

# StockPro

Sistema de gestión de inventario con arquitectura MVC

**Integrantes**
Andrés Felipe Guaca
Juan Pablo Cruz

17 de octubre de 2026

# 1. Descripción del proyecto

StockPro es una aplicación web para controlar el inventario de una organización: permite registrar productos, organizarlos por categorías, controlar entradas y salidas de stock y recibir alertas cuando una existencia está por agotarse. Cuenta con dos roles: **administrador** (gestiona todo el sistema) y **usuario** (consulta el catálogo y retira productos).

### Tecnologías

| Capa | Tecnología |
|---|---|
| Servidor | Node.js + Express 5 |
| Vistas | EJS + Bootstrap 5 + Bootstrap Icons |
| Base de datos | SQLite (módulo nativo de Node) |
| Gráficas | Chart.js |
| Seguridad | bcryptjs (hash de contraseñas) + express-session |
| Gestor de paquetes | pnpm |

# 2. Arquitectura MVC

El proyecto separa responsabilidades en tres capas, conectadas por un archivo de rutas:

**Modelo**
Datos y reglas de negocio. Consultas SQL y transacciones de stock.

**Vista**
Plantillas EJS que generan el HTML que ve el usuario.

**Controlador**
Recibe la petición, usa el modelo y elige la vista.

Estructura de carpetas

```
inventario-mvc/
├── package.json
├── pnpm-lock.yaml
├── public/css/style.css
└── src/
    ├── app.js                (configuración de Express)
    ├── seed.js               (datos de demostración)
    ├── models/               MODELO
    │   └── db.js  User.js  Product.js  Category.js  Movement.js
    ├── views/                VISTA (plantillas EJS)
    │   └── auth/  catalog/  admin/  partials/
    ├── controllers/          CONTROLADOR
    │   └── authController.js  catalogController.js  adminController.js
```

```
├── routes/index.js         (URL → controlador)
└── middleware/auth.js      (sesión y roles)
```

# 3. Ejemplo del flujo MVC (retirar un producto)

### Ruta — `src/routes/index.js`

```
// Usuario
router.get('/catalog', requireLogin, catalog.index);
router.get('/catalog/:id', requireLogin, catalog.show);
router.post('/catalog/:id/request', requireLogin, catalog.request);
router.get('/my-movements', requireLogin, catalog.mine);
```

### Controlador — `src/controllers/catalogController.js`

```
exports.request = (req, res) => {
  const quantity = parseInt(req.body.quantity, 10);
  const back = `/catalog/${req.params.id}`;
  if (!quantity || quantity < 1) {
    req.session.flash = { type: 'danger', text: 'Indica una cantidad válida.' };
    return res.redirect(back);
  }
  try {
    Movement.register({
      productId: req.params.id, userId: req.session.user.id,
      type: 'salida', quantity, note: req.body.note,
    });
    req.session.flash = { type: 'success', text: `Retiraste ${quantity} unidad(es)
correctamente.` };
    res.redirect('/my-movements');
  } catch (e) {
    req.session.flash = { type: 'danger', text: e.message };
    res.redirect(back);
  }
};
```

### Modelo — `src/models/Movement.js` (transacción que valida y descuenta el stock)

```
  // Registra el movimiento y actualiza el stock de forma atómica
  register({ productId, userId, type, quantity, note }) {
    return db.transaction(() => {
      const p = db.prepare('SELECT stock FROM products WHERE id = ?').get(productId);
      if (!p) throw new Error('El producto no existe.');
      if (type === 'salida' && quantity > p.stock) {
        throw new Error(`Stock insuficiente: solo hay ${p.stock} unidades disponibles.`);
      }
      const delta = type === 'entrada' ? quantity : -quantity;
      db.prepare('UPDATE products SET stock = stock + ? WHERE id = ?').run(delta, productId);
      return db.prepare('INSERT INTO movements (product_id,user_id,type,quantity,note) VALUES
(?,?,?,?,?)')
        .run(productId, userId, type, quantity, note || '').lastInsertRowid;
    });
  },
```

### Vista — `src/views/catalog/show.ejs` (formulario de retiro)

```
        <form method="post" action="/catalog/<%= product.id %>/request">
          <div class="mb-3"><label class="form-label">Cantidad a retirar</label>
            <input type="number" name="quantity" class="form-control" min="1" max="<%=
product.stock %>" value="1" required></div>
          <div class="mb-3"><label class="form-label">Motivo (opcional)</label>
            <input name="note" class="form-control" maxlength="120" placeholder="Ej: proyecto
de robótica"></div>
          <button class="btn btn-brand w-100"><i class="bi bi-bag-check"></i> Retirar
```

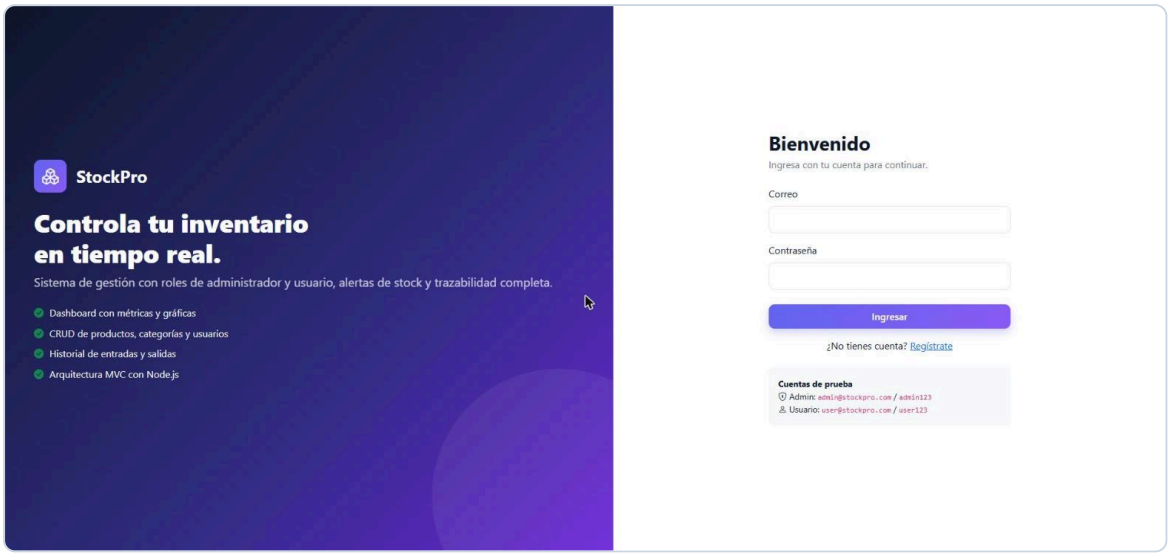
```
producto</button>
        </form>
```

# 4. Capturas del sistema

## 4.1 Acceso

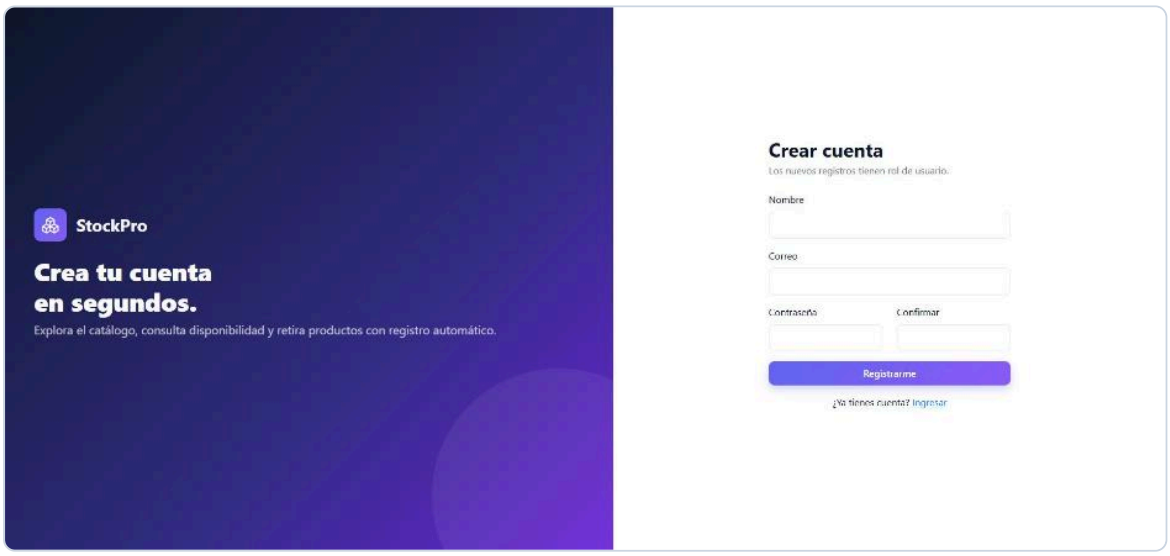
### Inicio de sesión

Pantalla de ingreso con las cuentas de prueba para administrador y usuario.




### Registro de usuarios

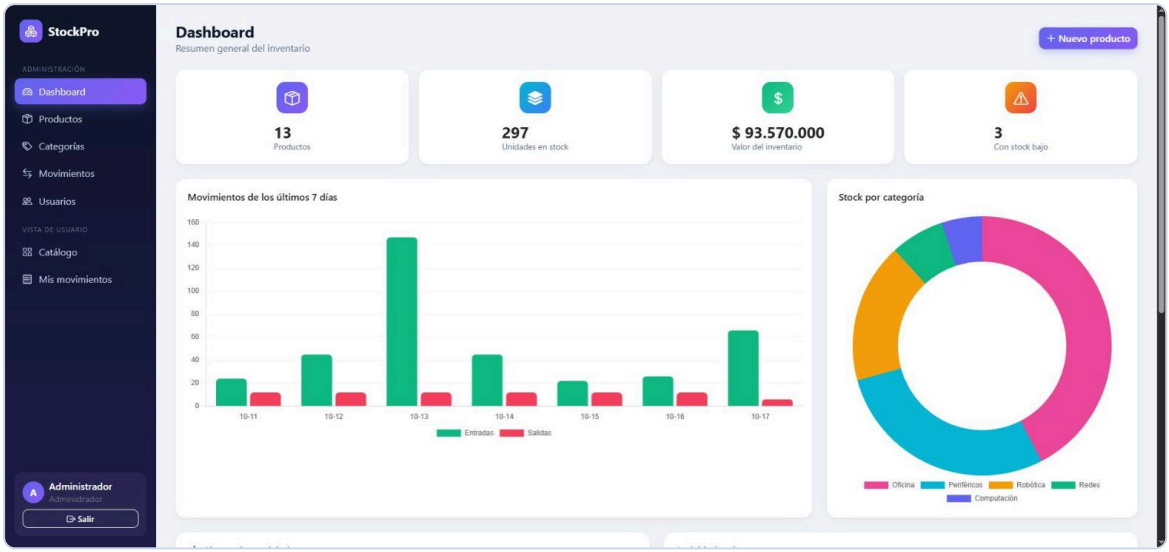
Los nuevos registros se crean con rol de usuario. Se valida nombre, correo, longitud y confirmación de contraseña.



## 4.2 Panel de administrador

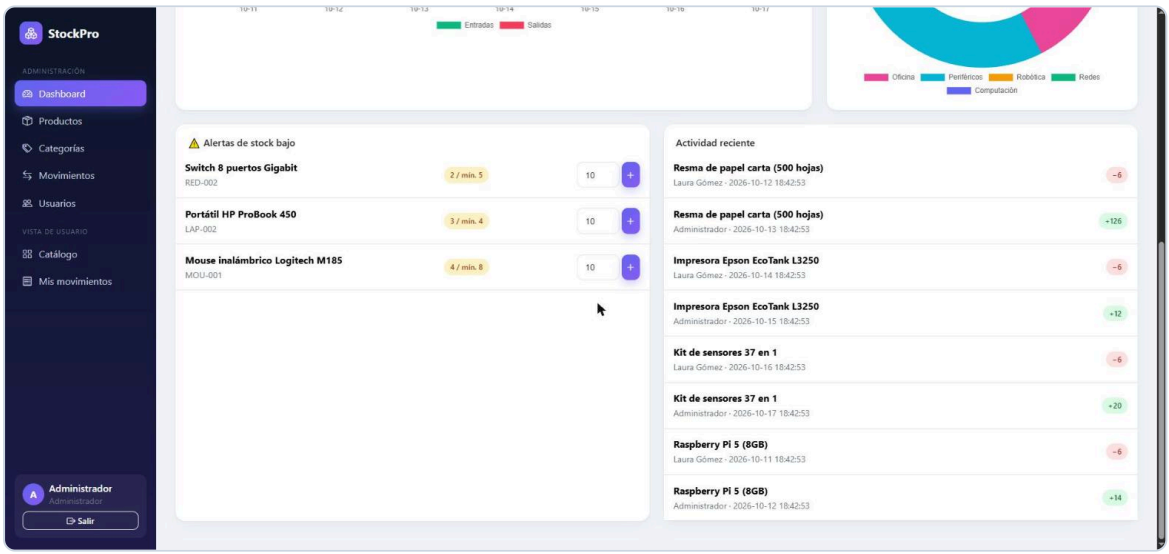
### Dashboard (métricas y gráficas)

Cuatro indicadores generales (productos, unidades, valor del inventario y alertas), gráfica de entradas/salidas de los últimos 7 días y distribución del stock por categoría.



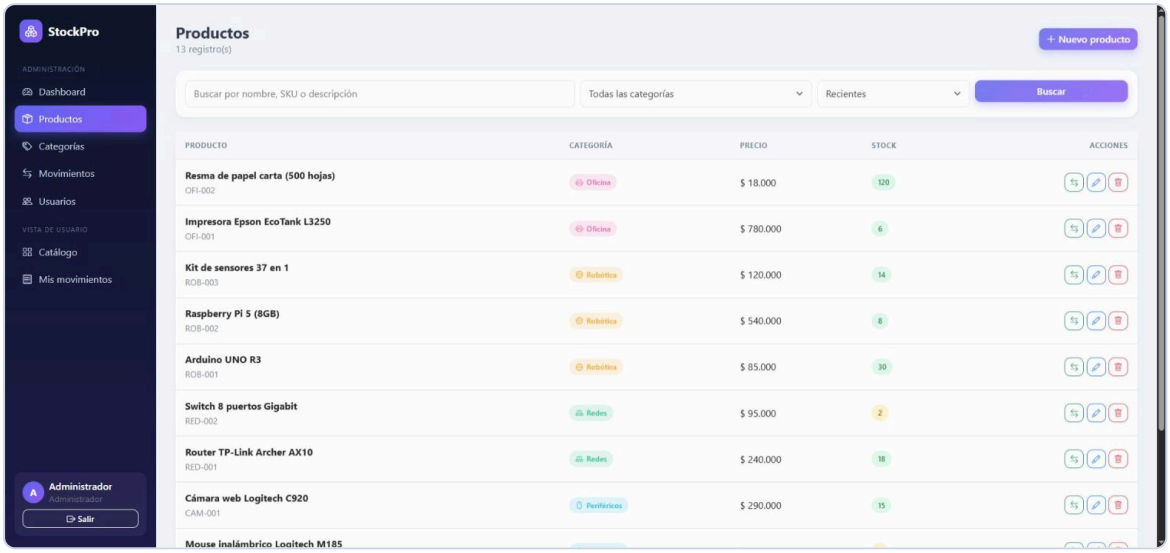

### Dashboard (alertas y actividad)

Alertas de stock bajo con reposición rápida y listado de actividad reciente.



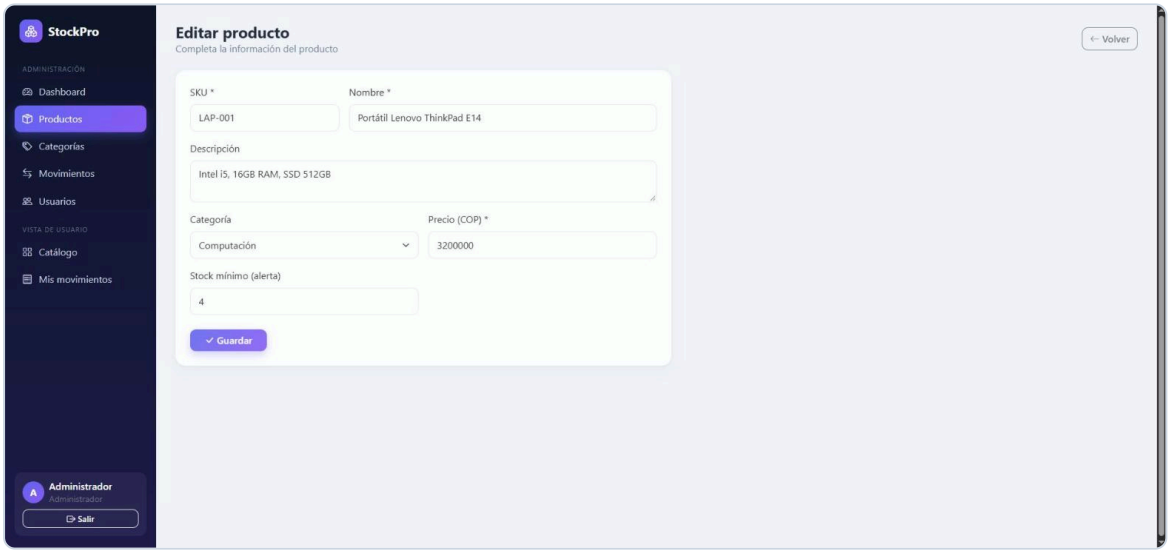
## Gestión de productos (listado)

Búsqueda, filtro por categoría, ordenamiento y paginación. Acciones: ajustar stock, editar y eliminar.



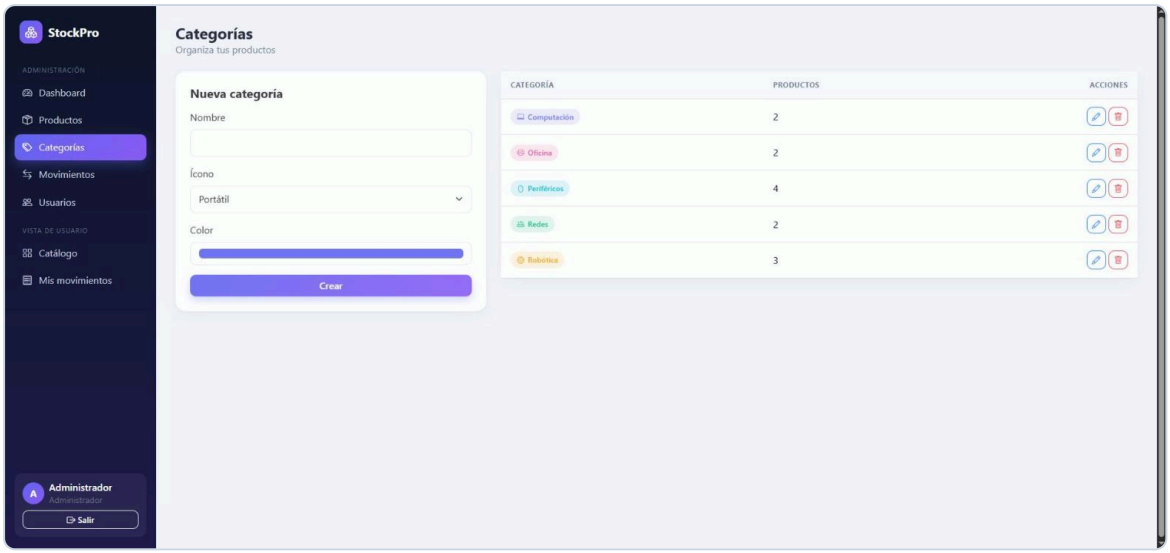

## Formulario de producto

Crear y editar productos con validación en el servidor.



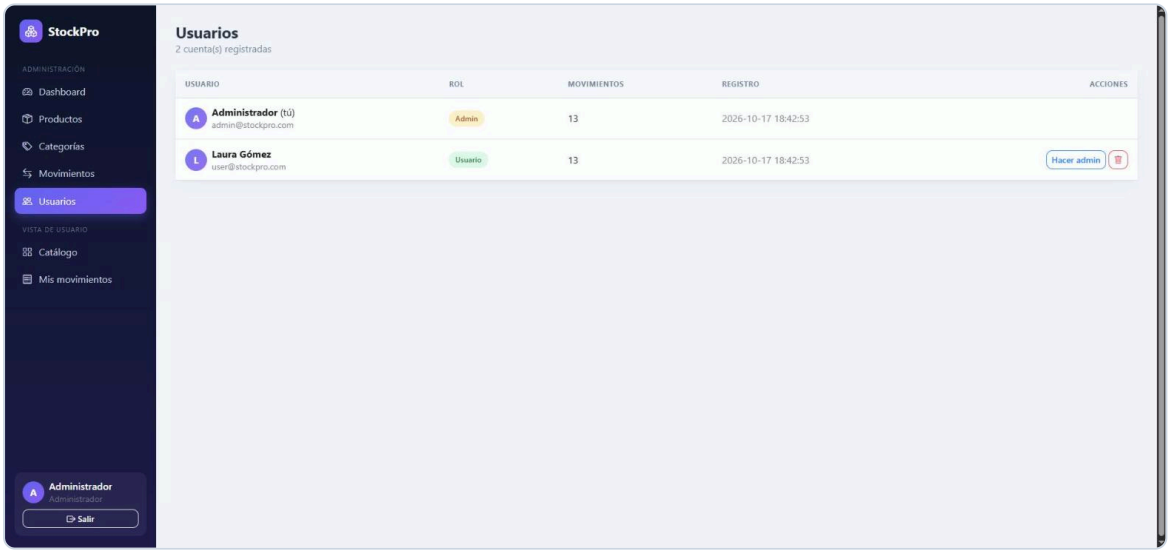
## Gestión de categorías

CRUD de categorías con ícono y color; se muestra cuántos productos tiene cada una.



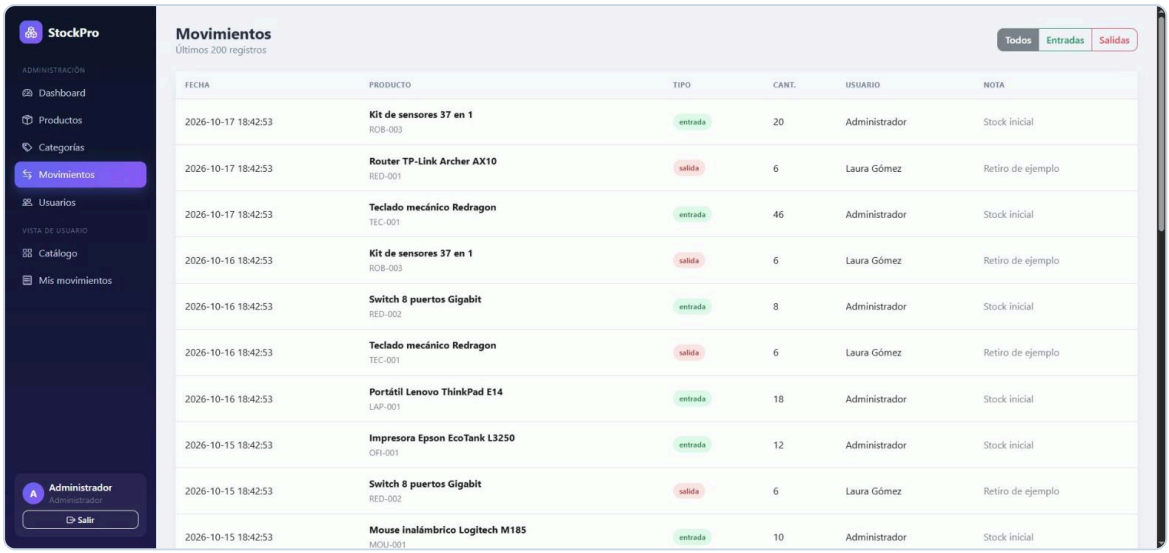

## Gestión de usuarios

El administrador puede cambiar el rol de una cuenta o eliminarla (no puede modificar su propia cuenta).



## Historial de movimientos

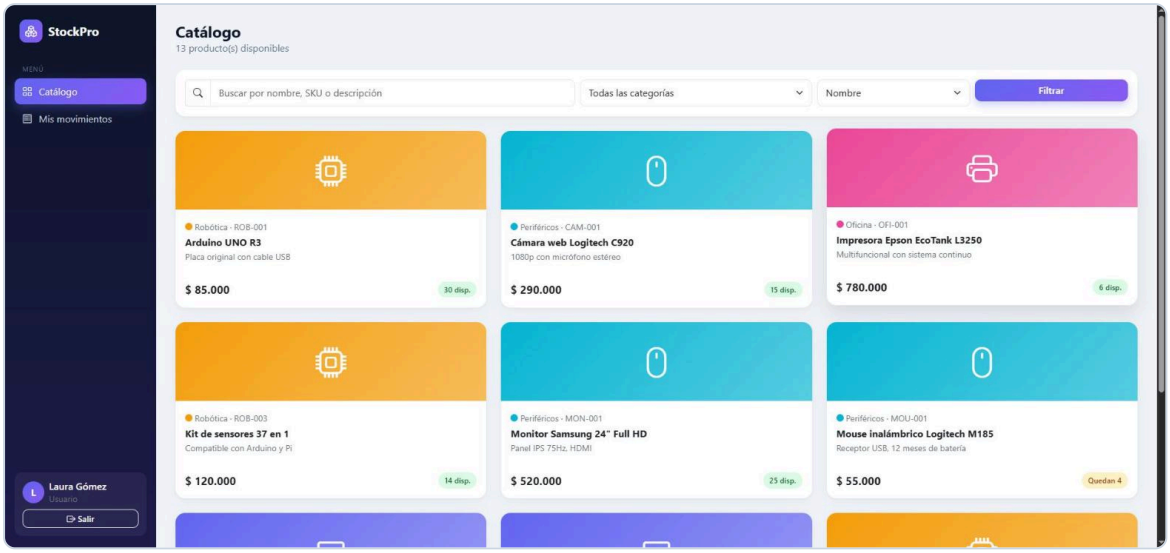
Trazabilidad de todas las entradas y salidas, con filtro por tipo, usuario y nota.



## 4.3 Vista de usuario

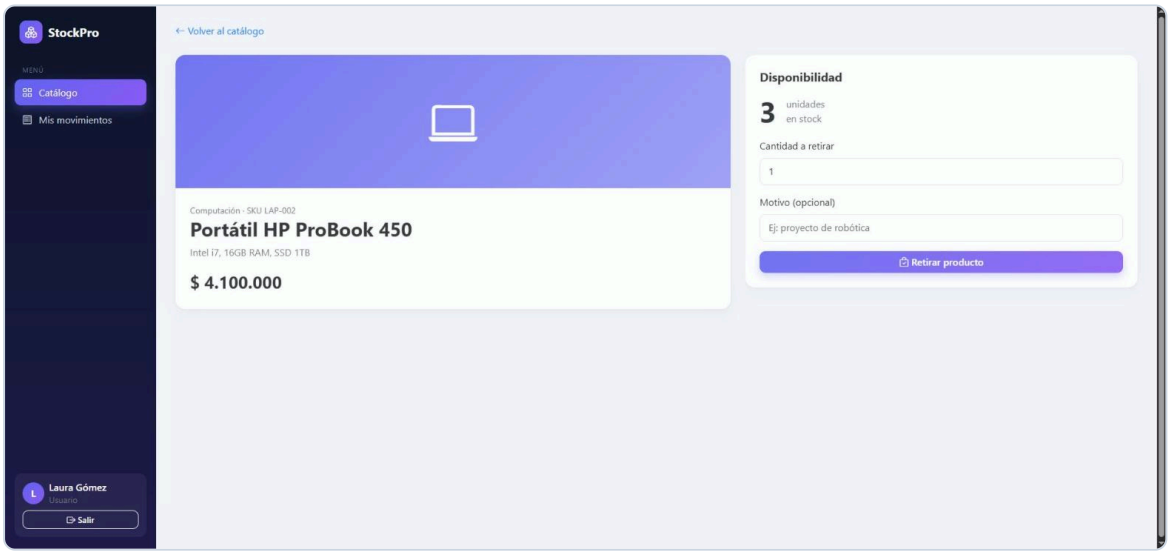
### Catálogo

Tarjetas de producto con categoría, precio y disponibilidad (disponible, pocas unidades, agotado), con búsqueda, filtros y paginación.



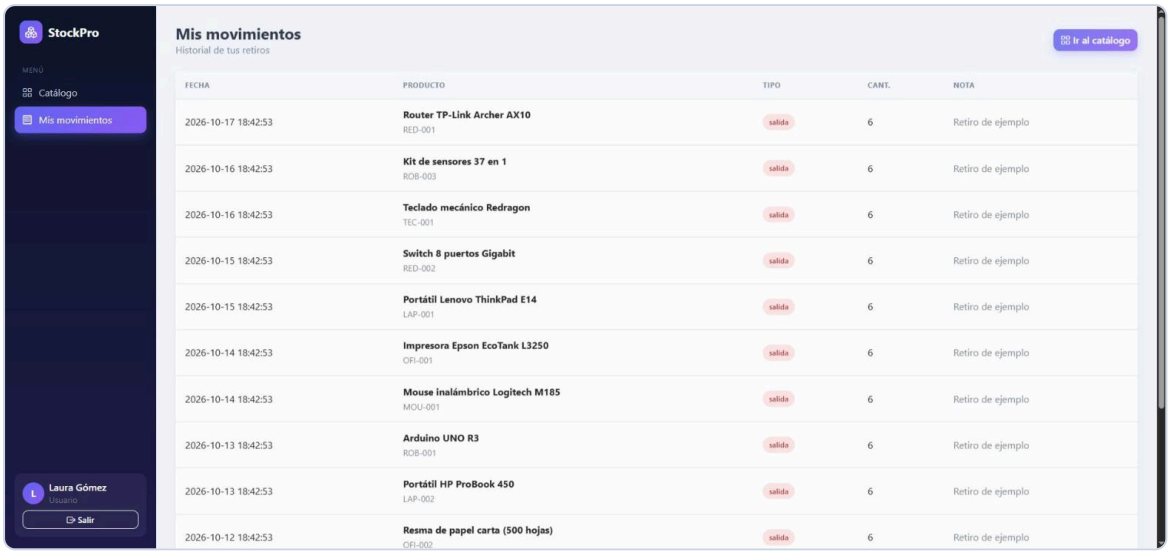

### Detalle y retiro de producto

El usuario ve el stock y solicita la cantidad a retirar; el sistema valida que no supere las existencias.



## Mis movimientos

Historial personal de retiros del usuario.




## Protección por roles

Si un usuario sin permisos intenta entrar a una ruta de administración (por ejemplo /admin/movements), el sistema responde 403 Acceso denegado.



## 5. Estado del avance

| Módulo | Estado |
|---|---|
| Autenticación y roles (admin / usuario) | Completo |
| CRUD de productos y categorías | Completo |
| Control de stock con entradas y salidas | Completo |
| Dashboard con gráficas y alertas | Completo |
| Gestión de usuarios | Completo |
| Catálogo y retiros para usuario | Completo |

### Seguridad implementada

- Contraseñas almacenadas con hash bcrypt.
- Rutas protegidas por sesión y por rol (el usuario recibe 403 en el panel admin).
- Consultas SQL parametrizadas (previene inyección SQL).
- Operaciones de stock en transacciones.

### Cuentas de prueba

| Rol | Correo | Contraseña |
|---|---|---|
| Administrador | admin@stockpro.com | admin123 |
| Usuario | user@stockpro.com | user123 |

### Cómo ejecutarlo

```
pnpm install
pnpm start      # http://localhost:3000
```